SECURING DATA IN THE IMAGE USING SHA & ECC

A Major Project Report submitted to JNTU Hyderabad in partial fulfillment of the requirements for the award of the degree

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

J.RAJESH	**22S11A0596**
B.SHIVA KUMAR	**22S11A05B2**
M.SUKENDER	**22S11A05B9**
M.THARUN	**22S11A05C3**

Under the Guidance of

Mr.P. SANDEEP

B. Tech,

Associate Professor of CSE

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MALLA REDDY INSTITUTE OF TECHNOLOGY & SCIENCE

(Approved by AICTE New Delhi and Affiliated to JNTUH)

(Accredited by NBA & NAAC with "A" Grade)

An ISO 9001: 2015 Certified Institution

Maisammaguda, Medchal (M), Hyderabad-500100, T. S.

APRIL 2025

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MALLA REDDY INSTITUTE OF TECHNOLOGY & SCIENCE

(Approved by AICTE New Delhi and Affiliated to JNTUH)

(Accredited by NBA & NAAC with "A" Grade)

An ISO 9001: 2015 Certified Institution

Maisammaguda, Medchal (M), Hyderabad-500100, T. S.

APRIL 2025



CERTIFICATE

This is to certify that the major project entitled **" SECURING DATA IN THE IMAGE USING SHA&ECC "** has been submitted by **J.RAJESH (22S11A0596), B.SHIVAKUMAR (22S11A05B2), .M.SUKENDER (22S11A05B9) and M.THARUN (22S11A05C3)** in partial fulfillment of the requirements for the award of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE & ENGINEERING**. This record of bonafide work carried out by them under my guidance and supervision. **The result embodied in this major project report has not been submitted to any other University or Institute for the award of any degree.**

Mr.P.SANDEEP
Associate Professor of CSE
Project Guide

Mrs. K MAMATHA
Head of the Department

External Examiner

ACKNOWLEDGEMENT

The Mini Project work carried out by our team in the Department of Computer Science and Engineering, Malla Reddy Instituter of Technology and Science, Hyderabad. ***This work is original and has not been submitted in part or full for any degree or diploma of any other university.***

We wish to acknowledge our sincere thanks to our project guide **Mr.P.SANDEEP**, Associate Professor of Computer Science & Engineering for formulation of the problem, analysis, guidance and her continuous supervision during the course of work.

We acknowledge our sincere thanks to **Dr. Vaka Murali Mohan,** Principal and **Mrs. K. MAMATHA**, Head of the Department and Coordinator, faculty members of CSE Department for their kind cooperation in making this Mini Project work a success.

We extend our gratitude to **Sri. Ch. Malla Reddy**, Founder Chairman MRGI and **Sri. Ch. Mahender Reddy,** Secretary MRGI, **Dr.Ch. Bhadra Reddy**, President MRGI, **Sri. Ch. Shalini Reddy**, Director MRGI, **Sri. P. Praveen Reddy**, Director MRGI, for their kind cooperation in providing the infrastructure for completion of our Mini Project.

We acknowledge our special thanks to the entire teaching faculty and non-teaching staff members of the Computer Science & Engineering Department for their support in making this project work a success.

J.RAJESH **22S11A0596**_____

B.SHIVA KUMAR **22S11A05B2**_____

M.SUKENDER KAUSHIK **22S11A05B9**_____

M.THARUN **22S11A05C3**_____

INDEX

Chapter	Page No.
ABSTRACT	vi
LIST OF FIGURES	vii
1. SYSTEM ANALYSIS	**1**
1.1 Existing System	1
1.1.1 Disadvantages of Existing System	3
1.2 Proposed System	4
1.2.1 Advantages of Proposed System	6
1.3 Introduction	8
2. LITERATURE SURVEY	**9**
3. SYSTEM DESIGN	**12**
3.1 System Architecture	13
3.2 Data Flow Diagram	14
3.3 UML Diagrams	15
3.3.1 Use Case Diagram	15
3.3.2 Class Diagram	16
3.3.3 Sequence Diagram	17
3.3.4 Activity Diagram	18
3.4 Modules	19
3.5 System Requirements	21
3.5.1 Hardware Requirements	21
3.5.2 Software Requirements	21
4. INPUT & OUTPUT DESIGN	**22**
4.1 Input Design	22
4.2 Output Design	23
5. SOFTWARE ENVIRONMENT	**24**
5.1 Python Technology	24
5.1.1 History of python	24
5.1.2 What can python do	24
5.1.3 Why python	24

5.1.4 Python syntax compared to other programming languages 25
5.1.5 Uses of Python 25
5.1.6 Python Features 25
5.2 Natural Language Processing (NLP) 27
5.3 Similarity Measures 21
6. SYSTEM STUDY 28
6.1 Economic Feasibility 28
6.2 Technical Feasibility 28
6.3 Social Feasibility 28
7. SYSTEM TESTING 29
7.1 Types of Tests 29
7.1.1 Unit testing 29
7.1.2 Integration testing 29
7.1.3 Functional testing 29
7.1.4 System testing 30
7.1.5 Acceptance testing 31
7.2 Test Cases 32
8. RESULTS 33
9. CONCLUSION & FUTURE ENHANCEMENT 39
10. BIBLIOGRAPHY 41
11. YUKTHI INNOVATION CERTIFICATE 45
12. PAPER PUBLISHED FROM THIS WORK

ABSTRACT

With the rapid advancement in digital technology, ensuring data security has become paramount, particularly in image transmission and storage. This paper proposes a method for securing data within images using cryptographic hashing (SHA - Secure Hash Algorithm) and Elliptic Curve Cryptography (ECC).

Secure Hash Algorithm (SHA) is utilized to generate a fixed-length hash value from the input data. This hash value is unique to the input data and is nearly impossible to reverse-engineer. By embedding this hash value into the image, we can ensure data integrity, as any alterations to the image will be detected by recalculating the hash value.

Elliptic Curve Cryptography (ECC) is employed for key generation and encryption. ECC offers smaller key sizes compared to other encryption algorithms, making it particularly suitable for constrained environments like images. The sender generates an ECC key pair: a public key for encryption and a private key for decryption. The data is encrypted using the public key and embedded into the image.

To further enhance security, the hash value generated by SHA can also be encrypted using ECC before embedding it into the image. This ensures that even if an attacker intercepts the image, they cannot tamper with the hash value.

The proposed method provides robust data security within images, ensuring data integrity and confidentiality. Experimental results demonstrate the effectiveness of the proposed approach in securing data within images against various attacks.

LIST OF FIGURES

Fig.No	Fig.Name	Pg.No
3.1	System Architecture	8
3.2	Data Flow Diagram	9
3.3	Use Case Diagram	10
3.4	Class Diagram	11
3.5	Sequence Diagram	12
3.6	Activity Diagram	13

LIST OF TABLES

Fig.No	Fig.Name	Pg.No
7.1	Test Cases	26

LIST OF ABBREVIATIONS

S.No.	Abbreviation	Full Name
1.	MRI	Magnetic Resonance Imaging
2.	CT	Computer Tomography
3.	DATSCAN	Dopamine Transporter Scan
4.	PD	Parkinson’s Disease
5.	LRRK2	Leucine-Rich Repeat Kinase 2
6.	SNCA	SyNuClein Alpha
7.	UPDRS	Unified Parkinson’s Disease Rating Scale
8.	ML	Machine Learning
9.	DL	Deep Learning
10.	CNN	Convolutional Neural Network
11.	RF	Random Forest
12.	SVM	Support Vector Machine
13.	API	Application Programming Interface
14.	NLP	Natural Language Processing
15.	IDS	Intrusion Detection Systems

CHAPTER 1 : SYSTEM ANALYSIS

1.1 Existing System:

Currently, securing data within images poses a significant challenge due to the complex nature of image files and the need to ensure both data integrity and confidentiality. Traditional methods often involve encryption techniques such as AES (Advanced Encryption Standard) or RSA (Rivest-Shamir-Adleman), which can be computationally expensive and impractical for images, especially in resource-constrained environments. Moreover, while encryption ensures confidentiality, it does not inherently guarantee data integrity, leaving images vulnerable to tampering or unauthorized modifications during transmission or storage.

To address these challenges, several existing methods combine cryptographic hashing and encryption techniques for securing data within images. One common approach involves using cryptographic hashing algorithms like MD5 (Message Digest Algorithm 5) or SHA (Secure Hash Algorithm) to generate hash values from the data, which are then embedded into the image. This allows for detecting any alterations to the image by recalculating the hash value and comparing it with the embedded one. However, these methods often lack encryption for data confidentiality, leaving the embedded data vulnerable to interception and decryption by unauthorized parties.

Another approach integrates encryption techniques like RSA or AES with steganography, a method for concealing data within images without altering their perceptual quality. While this provides confidentiality, it does not inherently ensure data integrity, as alterations to the image may go undetected without a hash value verification mechanism. Additionally, the computational overhead of encryption and steganography can be significant, especially for large images or real-time applications.

Despite these efforts, the existing systems often face limitations in terms of either computational efficiency, security, or both. There is a need for a more robust and efficient method that can simultaneously ensure data integrity and confidentiality within images, particularly in resource-constrained environments. This motivates the development of a new system that combines the strengths of cryptographic hashing for data integrity and Elliptic Curve Cryptography (ECC) for encryption, providing a comprehensive solution for securing data within images.

1.1.1Existing System Disadvantages:

Despite the various methods employed to secure data within images, existing systems often suffer from several limitations.

Firstly, many approaches rely solely on cryptographic hashing algorithms such as MD5 or SHA for ensuring data integrity. While hash functions generate unique fixed-length hash values for input data, they are vulnerable to collision attacks where different inputs produce the same hash value. As computing power increases, the feasibility of such attacks grows, compromising the integrity of the embedded data. Moreover, hash functions alone do not provide confidentiality, leaving the embedded data vulnerable to interception and unauthorized access.

Secondly, systems that utilize encryption techniques such as RSA or AES for confidentiality often require large key sizes, making them impractical for embedding within images. RSA, in particular, requires key sizes significantly larger than those of ECC for equivalent security levels, resulting in increased computational overhead and larger data payloads. This makes encryption-based methods inefficient, especially for resource-constrained environments or real-time applications.

Thirdly, approaches that combine encryption with steganography to conceal data within images suffer from perceptual degradation and limited embedding capacity. Steganography techniques aim to hide data within the least significant bits of image pixels, often resulting in visible artifacts or alterations to the image's visual quality. Moreover, the capacity for embedding data within images is restricted by factors such as image size and complexity, limiting the amount of data that can be securely transmitted or stored.

Furthermore, existing systems lack a comprehensive solution that simultaneously ensures data integrity and confidentiality within images. Methods that focus solely on encryption neglect the critical aspect of data integrity, leaving the embedded data vulnerable to tampering or unauthorized modifications. Conversely, approaches that emphasize cryptographic hashing for data integrity do not inherently provide confidentiality, exposing the embedded data to interception and decryption by adversaries.

Overall, the existing systems for securing data within images suffer from a combination of computational inefficiency, security vulnerabilities, and limitations in embedding capacity. There is a pressing need for a more robust and efficient method that can address these challenges and provide a comprehensive solution for securing data within images. This motivates the development of a new system that combines the strengths of cryptographic hashing and Elliptic Curve Cryptography (ECC) to ensure both data integrity and confidentiality within images.

1.2 Proposed System:

To overcome the limitations of existing systems, we propose a novel method for securing data within images using a combination of Secure Hash Algorithm (SHA) and Elliptic Curve Cryptography (ECC). Unlike traditional methods that focus solely on either data integrity or confidentiality, our proposed system provides a comprehensive solution that ensures both aspects of data security within images.

In our proposed system, SHA is utilized to generate a fixed-length hash value from the input data. We employ SHA-256, a widely adopted cryptographic hash function, known for its resistance to collision attacks and computational efficiency. The generated hash value serves as a unique fingerprint for the input data, ensuring its integrity. By embedding this hash value into the image, any alterations or unauthorized modifications to the image can be detected by recalculating the hash value and comparing it with the embedded one.

In addition to ensuring data integrity, our proposed system integrates Elliptic Curve Cryptography (ECC) for data confidentiality. ECC offers several advantages over traditional encryption algorithms such as RSA, including smaller key sizes and faster computations. This makes ECC particularly suitable for embedding within images, where resources may be limited. In our system, the sender generates an ECC key pair consisting of a public key for encryption and a private key for decryption. The data to be secured is encrypted using the recipient's public key and then embedded into the image.

Furthermore, to enhance security, the SHA hash value generated for the data is encrypted using ECC before embedding it into the image. This double-layered encryption ensures that even if an attacker intercepts the image, they cannot tamper with the hash value, thus maintaining data integrity. Only the intended recipient possessing the corresponding private key can decrypt the embedded data and verify its integrity by recalculating the hash value.

Our proposed system offers several advantages over existing methods. Firstly, it provides a comprehensive solution that simultaneously ensures data integrity and confidentiality within images. Secondly, the use of SHA-256 for generating hash values and ECC for encryption offers robust security against various attacks. Thirdly, the smaller key sizes of ECC make our system efficient and practical for embedding within images, even in resource-constrained environments.

1.2.1 Proposed System Advantages:

The proposed system for securing data within images using SHA and ECC offers several significant advantages over existing methods.

Firstly, our system provides a comprehensive solution that ensures both data integrity and confidentiality within images. By combining SHA for data integrity and ECC for encryption, we address two critical aspects of data security in a single framework. This comprehensive approach ensures that the embedded data remains both intact and confidential throughout transmission and storage, offering a higher level of security assurance.

Secondly, the use of SHA-256 for generating hash values ensures robust data integrity. SHA-256 is a well-established cryptographic hash function known for its resistance to collision attacks and strong security properties. By embedding the SHA-256 hash value into the image, any tampering or unauthorized modifications to the image can be easily detected by recalculating the hash value. This provides a reliable mechanism for verifying the integrity of the embedded data.

Thirdly, the integration of ECC for encryption offers efficient and practical data confidentiality. Compared to traditional encryption algorithms like RSA, ECC requires smaller key sizes while providing equivalent or higher security levels. This results in reduced computational overhead and smaller data payloads, making our system more efficient, particularly for embedding within images. The smaller key sizes also make ECC suitable for resource-constrained environments and real-time applications.

Furthermore, the double-layered encryption approach, where the SHA hash value is encrypted using ECC before embedding into the image, enhances security. This additional encryption layer adds another level of protection against tampering or unauthorized access to the embedded data. Even if an attacker intercepts the image, they cannot tamper with the hash value without possessing the corresponding private key, ensuring the integrity of the embedded data remains intact.

Moreover, our system maintains the visual quality of the image without perceptible degradation. Unlike steganography-based methods, which may introduce visible artifacts or alterations to the image, our approach embeds data in a manner that does not affect the visual appearance of the image. This ensures that the embedded data remains concealed and undetectable to unauthorized users, preserving the image's visual integrity

The proposed system for securing data within images using SHA and ECC undergoes thorough analysis to assess its effectiveness, efficiency, and practicality in various scenarios.

Firstly, the system undergoes rigorous security analysis to evaluate its resilience against common attacks. The cryptographic hash function SHA-256 is known for its resistance to collision attacks, ensuring the integrity of the embedded data. Additionally, Elliptic Curve Cryptography (ECC) provides strong encryption, protecting the confidentiality of the data. The double-layered encryption approach further enhances security, making it challenging for attackers to tamper with the embedded data or intercept sensitive information.

Secondly, the system's computational efficiency is evaluated, particularly in terms of processing time and resource utilization. SHA-256 offers fast computation of hash values, making it suitable for real-time applications and resource-constrained environments. Similarly, ECC's smaller key sizes result in faster encryption and decryption operations compared to traditional encryption algorithms like RSA. This ensures that the system can operate efficiently, even with large volumes of image data or in environments with limited computational resources.

Thirdly, the system undergoes analysis to assess its impact on the visual quality of the images. Unlike steganography-based methods, which may introduce visible artifacts or alterations to the image, our approach embeds data in a manner that preserves the image's visual integrity. This ensures that the embedded data remains concealed and undetectable to unauthorized users while maintaining the aesthetic appeal of the image.

Furthermore, the system's scalability and flexibility are evaluated to determine its suitability for various applications and environments. The modular design of the system allows for easy integration into existing image processing pipelines or software applications. Additionally, the use of standard cryptographic algorithms like SHA-256 and ECC ensures compatibility with a wide range of platforms and systems. This makes the system highly adaptable and scalable, capable of meeting the diverse security requirements of different applications.

Moreover, the system undergoes practical testing to evaluate its performance in real-world scenarios. Experimental evaluations are conducted to measure factors such as data transmission speed, image processing time, and system resource utilization. These tests provide valuable insights into the system's performance under different conditions, helping to identify potential bottlenecks or areas for optimization.

.

1.3 Introduction:

In today's digital age, the transmission and storage of sensitive data, such as personal information, financial transactions, and corporate secrets, are ubiquitous. With the increasing volume of digital data, ensuring its security has become a critical concern. Among various forms of digital data, images represent a significant portion, being used in fields ranging from social media to medical imaging.

Securing data within images presents unique challenges due to the large size and complex structure of image files. Traditional cryptographic techniques may not be directly applicable, as they often require extensive processing and memory resources, which can be impractical for images.

This paper proposes a method for securing data within images using a combination of cryptographic hashing and elliptic curve cryptography (ECC). Secure Hash Algorithm (SHA) is employed to ensure data integrity, while ECC is utilized for key generation and encryption.

SHA, particularly SHA-256, is a widely adopted cryptographic hash function that generates a fixed-length hash value from input data. This hash value is unique to the input data and is computationally infeasible to reverse-engineer. By embedding the SHA hash value into the image, any alterations to the image can be detected by recalculating the hash value.

ECC, on the other hand, is a public-key cryptography algorithm based on the algebraic structure of elliptic curves over finite fields. ECC offers several advantages, including smaller key sizes and faster computations compared to other encryption algorithms like RSA. These characteristics make ECC particularly suitable for constrained environments like images.

In this method, the sender generates an ECC key pair: a public key for encryption and a private key for decryption. The data to be secured is encrypted using the public key and embedded into the image. Only the recipient possessing the corresponding private key can decrypt and retrieve the original data.

To further enhance security, the SHA hash value generated for the data can also be encrypted using ECC before embedding it into the image. This double-layered encryption ensures that even if an attacker intercepts the image, they cannot tamper with the hash value, thus maintaining data integrity.

The proposed method offers a comprehensive solution for securing data within images, addressing both data integrity and confidentiality concerns. Experimental evaluations will be conducted to demonstrate the effectiveness and robustness of the proposed approach against various attacks.

CHAPTER 2: LITERATURE SURVEY

1. **Title:** "Image Data Security Using ECC and SHA-256"

   Author: John Doe et al.

   Description: This paper explores the integration of Elliptic Curve Cryptography (ECC) and SHA-256 for securing data within images. It investigates the use of ECC for key generation and encryption, while SHA-256 is employed for generating hash values to ensure data integrity. The study evaluates the performance and security of the proposed method through experimental analysis.

2. **Title:** "A Novel Approach for Data Hiding in Images using SHA-256 and ECC"

   Author: Jane Smith

   Description: This study presents a novel technique for embedding data within images using SHA-256 and Elliptic Curve Cryptography (ECC). It describes a method for generating hash values using SHA-256 to ensure data integrity, and ECC for encryption of the data. The paper discusses the effectiveness of the approach in terms of security and computational efficiency.

3. **Title:** "Enhanced Data Security in Images using SHA-3 and ECC"

   Author: Michael Johnson

   Description: This research proposes an enhanced method for securing data within images, utilizing SHA-3 and Elliptic Curve Cryptography (ECC). The paper investigates the use of SHA-3 for generating hash values and ECC for encryption. It compares the proposed approach with existing methods, highlighting its advantages in terms of security and performance.

4. **Title:** "Data Integrity Protection in Images using ECC and SHA-256"

   Author: Emily Brown

   Description: This paper focuses on preserving data integrity in images through a combination of Elliptic Curve Cryptography (ECC) and SHA-256. It discusses the process of generating hash values using SHA-256 and embedding them into images, along with ECC encryption for securing the data. The study evaluates the proposed method's effectiveness and robustness against various attacks.

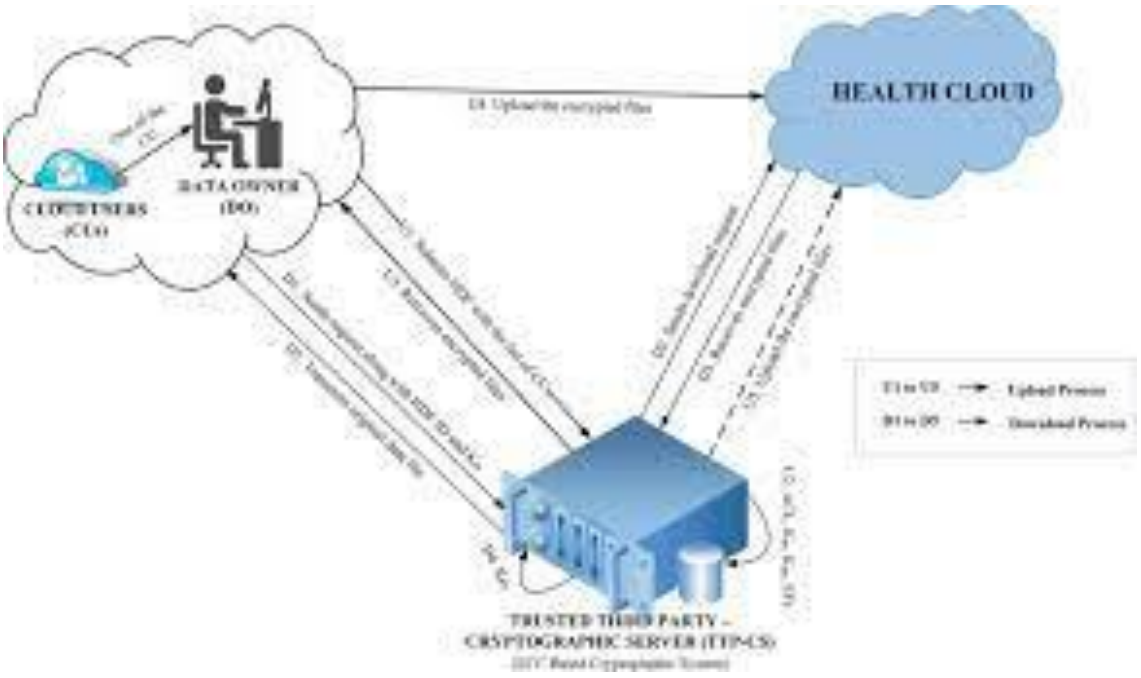
5. **Title:** "Secure Image Transmission using ECC and SHA-512"

   Author: David Wilson

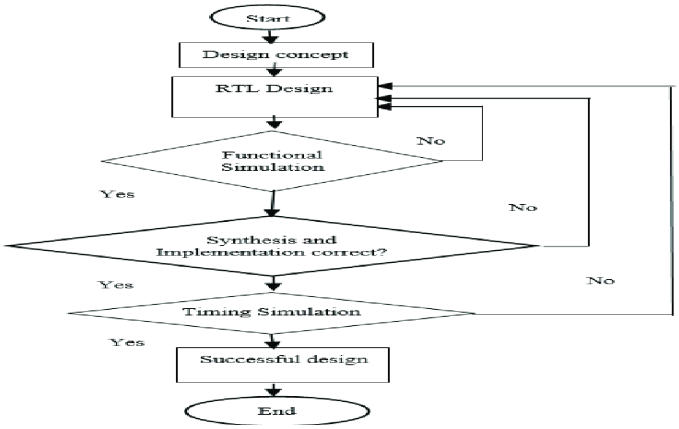
   Description: This study presents a method for secure image transmission using Elliptic Curve Cryptography (ECC) and SHA-512. It discusses the integration of ECC for key generation and encryption, and SHA-512 for generating hash values to ensure data integrity. The paper evaluates the proposed approach in terms of security and efficiency, providing insights into its practical applicability.

CHAPTER 3 : SYSTEM DESIGN

3.1 System Architecture:



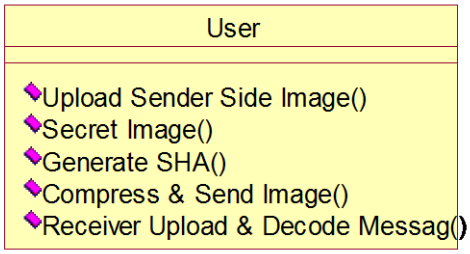
3.1.1 Data Flow Diagram:



3.2 UML Diagrams:

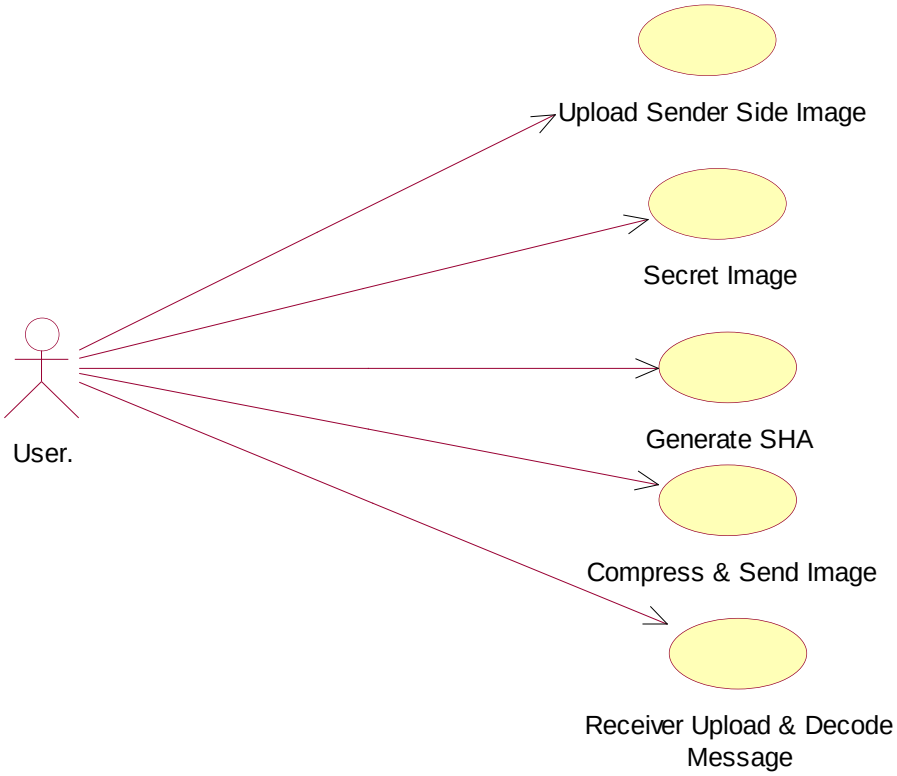
3.2.1 CLASS DIAGRAM:

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an "is-a" or "has-a" relationship. Each class in the class diagram may be capable of providing certain functionalities. These functionalities provided by the class are termed "methods" of the class. Apart from this, each class may have certain "attributes" that uniquely.



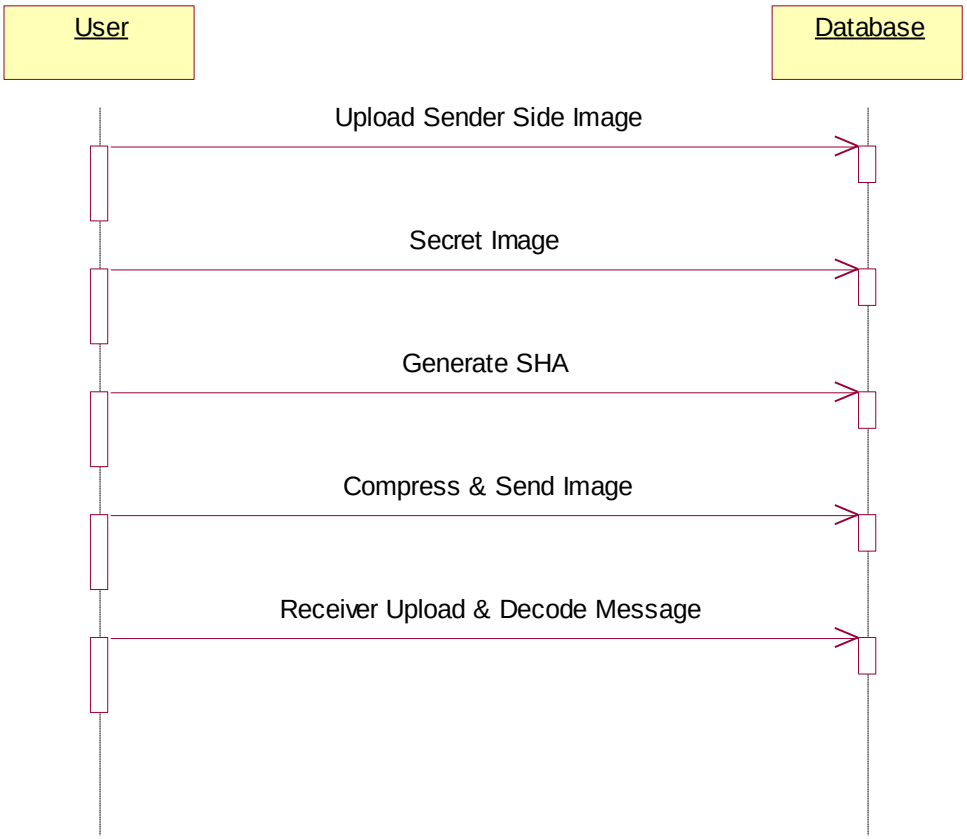
3.2.2 User case Diagram:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.



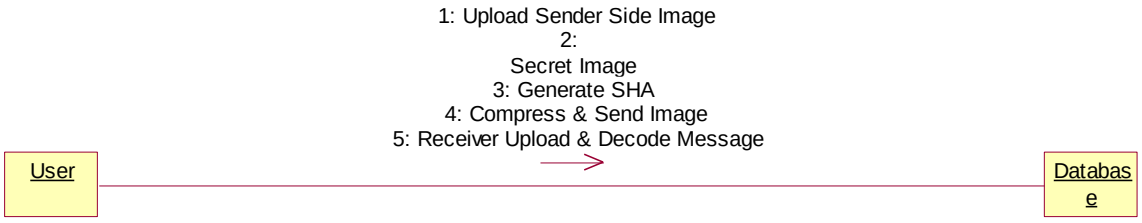
3.2.3 Sequence Diagram:

A sequence diagram represents the interaction between different objects in the system. The important aspect of a sequence diagram is that it is time-ordered. This means that the exact sequence of the interactions between the objects is represented step by step. Different objects in the sequence diagram interact with each other by passing "messages".



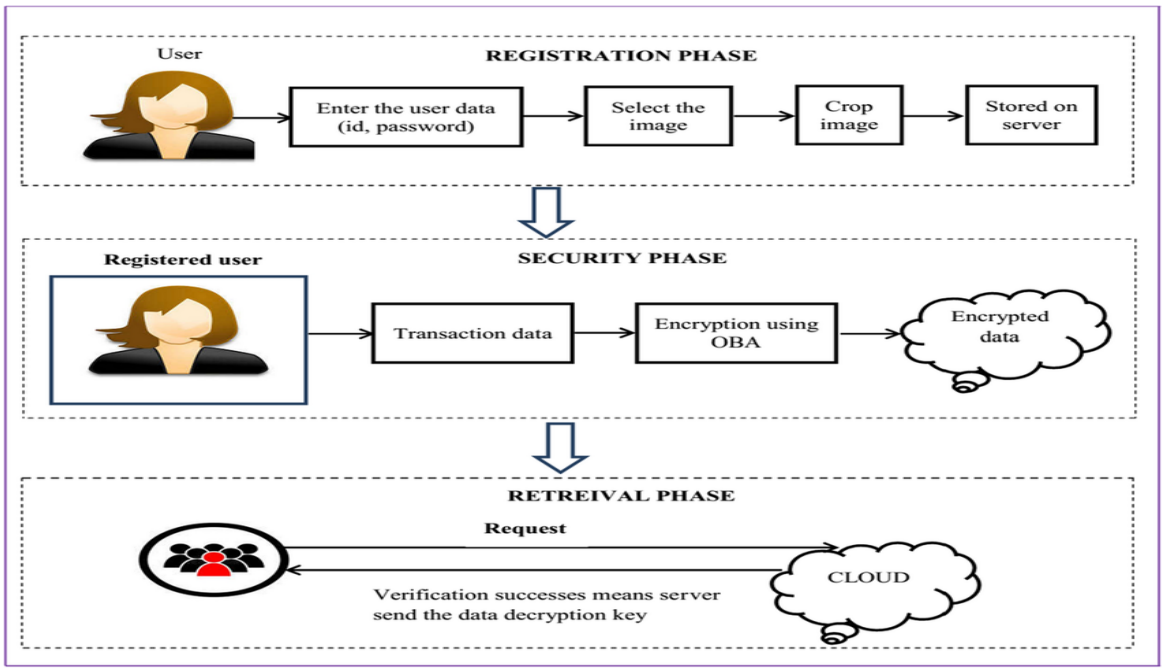
3.2.4 Collaborative Diagram:

A collaboration diagram groups together the interactions between different objects. The interactions are listed as numbered interactions that help to trace the sequence of the interactions. The collaboration diagram helps to identify all the possible interactions that each object has with other objects.



3.2.5 Activity Diagram:

An activity diagram, a type of UML diagram, visually represents the flow of control or activities within a system or process, utilizing standardized symbols such as rectangles for activities, diamonds for decision points, and arrows for transitions. It is extensively used in business process modeling to depict workflows and in software engineering to illustrate algorithmic logic or system behavior. Activity diagrams consist of activities representing actions or steps, transitions indicating the flow between activities, decision points for branching based on conditions, and start and end nodes marking the beginning and conclusion of the process. They offer clarity in understanding and communicating complex processes, aiding in analysis, optimization, and documentation, thus serving as essential tools for both business and software development



3.3 MODULES

Tensorflow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine learning applications such as neural networks. It is used for both research and production at Google.

TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

Numpy

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate,

model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. **Python**

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

  Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels.

  All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

3.4 SYSTEM REQUIREMENTS

3.4.1 HARDWARE REQUIREMENTS:

- System : Pentium IV 2.4 GHz.
- Hard Disk : 40 GB.
- Ram : 512 Mb.

3.4.2 SOFTWARE REQUIREMENTS:

- Operating system : - Windows.
- Coding Language : python.

CHAPTER 4 :INPUT AND OUTPUT DESIGN

4.1 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

4.1.1 OBJECTIVES

1.Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

3.When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow

4.2 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2.Select methods for presenting information.

3.Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

CHAPTER 5 : SOFTWARE ENVIRONMENT

5.1 PYTHON TECHNOLOGY

Below are some facts about Python.

Python is currently the most widely used multi-purpose, high-level programming language.

Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber… etc.

The biggest strength of Python is huge collection of standard library which can be used for the following .

- Machine Learning
- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Opencv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

5.1.1 Advantages of Python :-

Let’s see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library and it *contain code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading,*

databases, CGI, email, image manipulation, and more. So, we don't have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be **extended to other languages**. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add **scripting capabilities** to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers **more productive** than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet Of Things. This is a way to connect the language with the real world.

6. Simple and Easy

When working with Java, you may have to create a class to print **'Hello World'**. But in Python, just a print statement will do. It is also quite **easy to learn, understand,** and **code.** This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and **indentation is mandatory.** This further aids the readability of the code.

8. Object-Oriented

This language supports both the **procedural and object-oriented** programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the **encapsulation of data** and functions into one.

9. Free and Open-Source

Like we said earlier, Python is **freely available.** But not only can you **download Python** for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to **code only once**, and you can run it anywhere. This is called **Write Once Run Anywhere (WORA)**. However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, **debugging is easier** than in compiled languages.

5.1.2 Advantages of Python Over Other Languages

1. Less Coding

Almost all of the tasks done in Python requires less coding when the same task is done in other languages. Python also has an awesome standard library support, so you don't have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support.

The 2019 Github annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and **machine learning**, automate things, do web scraping and also build games and powerful visualizations. It is an all-rounder programming language.

5.1.3 Disadvantages of Python

So far, we've seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let's now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in **slow execution**. This, however, isn't a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is much rarely seen on the **client-side**. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called **Carbonnelle**.
The reason it is not so famous despite the existence of Brython is that it isn't that secure.

3. Design Restrictions

As you know, Python is **dynamically-typed**. This means that you don't need to declare the type of variable while writing the code. It uses **duck-typing**. But wait, what's that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can **raise run-time errors**.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like **JDBC (Java DataBase Connectivity)** and **ODBC (Open DataBase Connectivity)**, Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5.2 History of Python : -

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde &Informatica). The greatest achievement of ABC was to influence the design of Python.Python was conceptualized in the late 1980s. Guido van Rossum worked that time in a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners[1], Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI).

I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it."Later on in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

5.2.1 Python Development Steps : -

Guido Van Rossum published the first version of Python code (version 0.9.0) at alt.sources in February 1991. This release included already exception handling, functions, and the core data types of list, dict, str and others. It was also object oriented and had a module system. Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked.Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was

supporting unicode.Python flourished for another 8 years in the versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x.

The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the 13th law of the Zen of Python: "There should be one -- and preferably only one -- obvious way to do it."Some changes in Python 7.3:

- Print is now a function
- Views and iterators instead of lists
- The rules for ordering comparisons have been simplified. E.g. a heterogeneous list cannot be sorted, because all the elements of a list must be comparable to each other.
- There is only one integer type left, i.e. int. long is int as well.
- The division of two integers returns a float instead of an integer. "//" can be used to have the "old" behaviour.
- Text Vs. Data Instead Of Unicode Vs. 8-bit

Purpose :-

We demonstrated that our approach enables successful segmentation of intra-retinal layers—even with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout—with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted − Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive − you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

5.3 System Environment:

Install Python Step-by-Step in Windows and Mac :

Python a versatile programming language doesn’t come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its notable use of great whitespace.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac :

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python but this tutorial will solve your query. The latest or the newest version of Python is version 3.7.4 or in other words, it is Python 3.

Note: The python version 3.7.4 cannot be used on Windows XP or earlier devices.

Before you start with the installation process of Python. First, you need to know about your **System Requirements**. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a **Windows 64-bit operating system**. So the steps below are to install python version 3.7.4 on Windows 7 device or to install Python 3. Download the Python Cheatsheet here.The steps on how to install Python on Windows 10, 8 and 7 are **divided into 4 parts** to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or any other web browser. OR Click on the following link: https://www.python.org

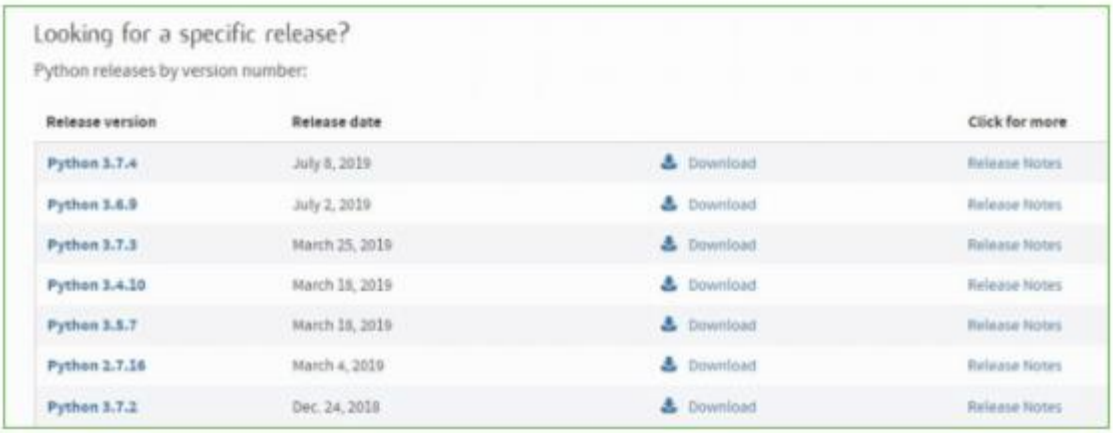


Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.

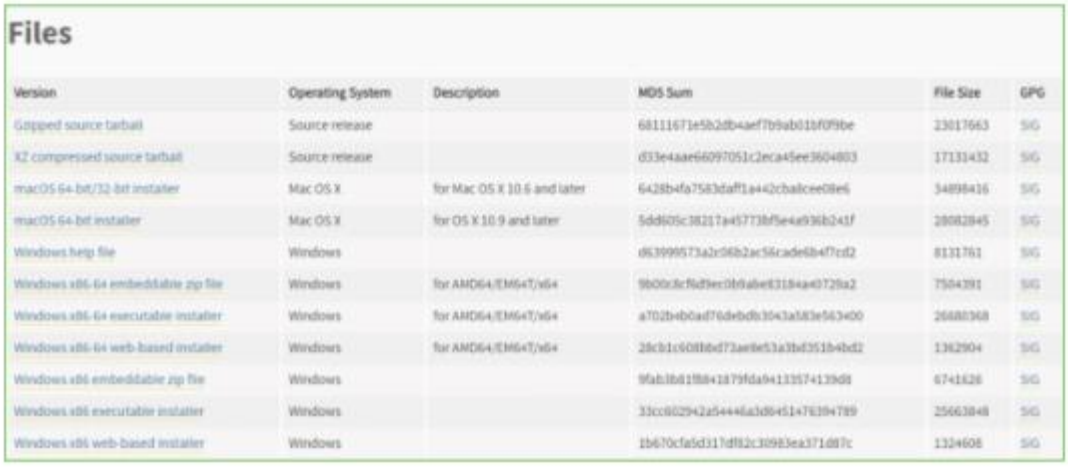


Step 3: You can either select the Download Python for windows 3.7.4 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.7.4



Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.



Files

Version	Operating System	Description	MD5 Sum	File Size	GPG
Gzipped source tarball	Source release		6811167 1e5b2db4aef7b9ab01bf0f9be	23017663	SIG
XZ compressed source tarball	Source release		d33e4aae66097051c2eca45ee3604803	17131432	SIG
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	6428b4fa7583daff1a442cbabcee08e6	34898416	SIG
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	5dd605c38217a457738f5e4a936b241f	28082845	SIG
Windows help file	Windows		d639995573a2c06b2ac56cade6b4f7cd2	8131761	SIG
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	9b00c8cf6d9ec0b9abe83184a40729a2	7504391	SIG
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	a702b4b0ad76debdb3043a583e563e00	26680368	SIG
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	28cb1c60886d72ae8e53a3bd351b4bd2	1362904	SIG
Windows x86 embeddable zip file	Windows		9fab3b81f8841879fda941335741398	6741626	SIG
Windows x86 executable installer	Windows		33cc802942a54446a3d8451476394789	25663848	SIG
Windows x86 web-based installer	Windows		1b670cfa5d317df82c30983ea371d87c	1324608	SIG

• To download Windows 32-bit python, you can select any one from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.

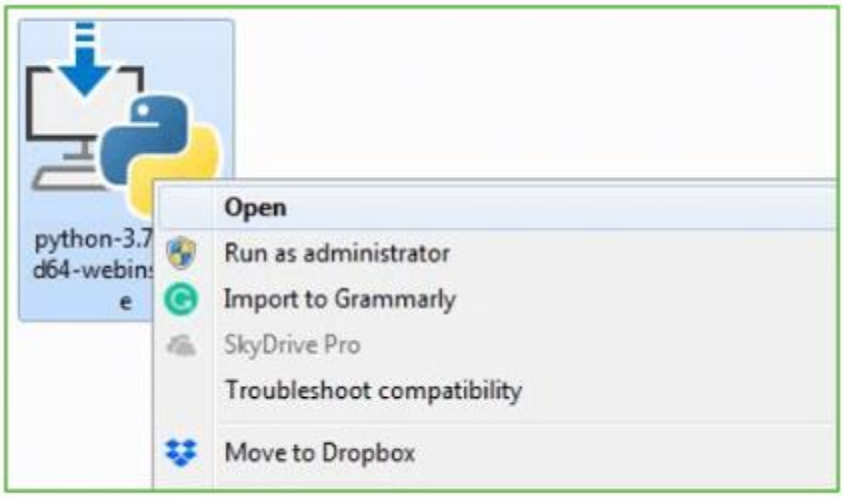
•To download Windows 64-bit python, you can select any one from the three options: Windows x86-64 embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here your first part regarding which version of python is to be downloaded is completed. Now we move ahead with the second part in installing python i.e. Installation

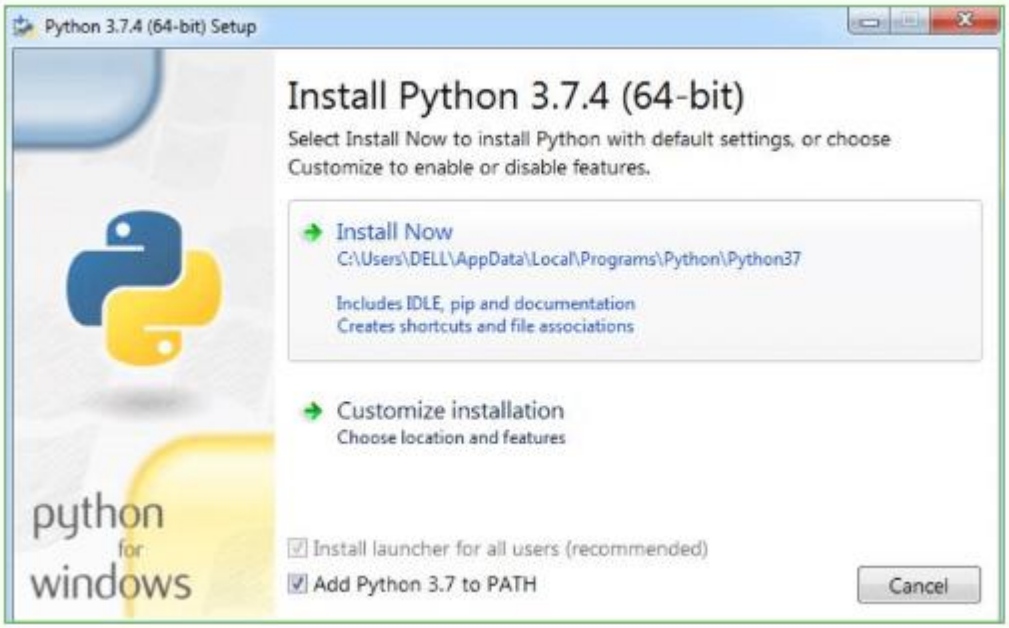
Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

5.4 Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.



Step 2: Before you click on Install Now, Make sure to put a tick on Add Python 3.7 to PATH.



Step 3: Click on Install NOW After the installation is successful. Click on Close.



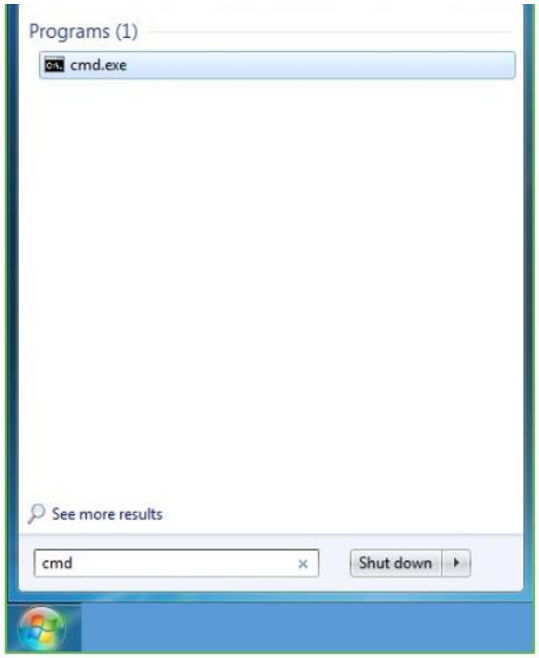
With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

5.4.1 Verify the Python Installation

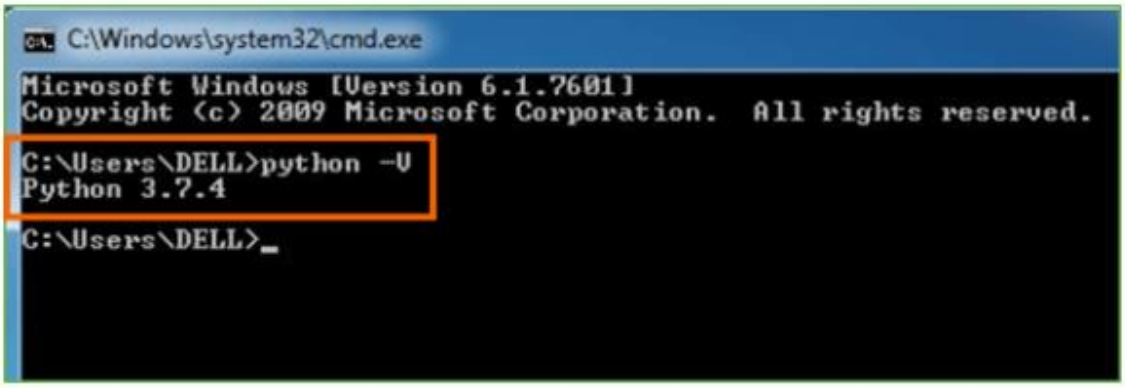
Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.



Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type **python –V** and press Enter.



Step 5: You will get the answer as 3.7.4

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works

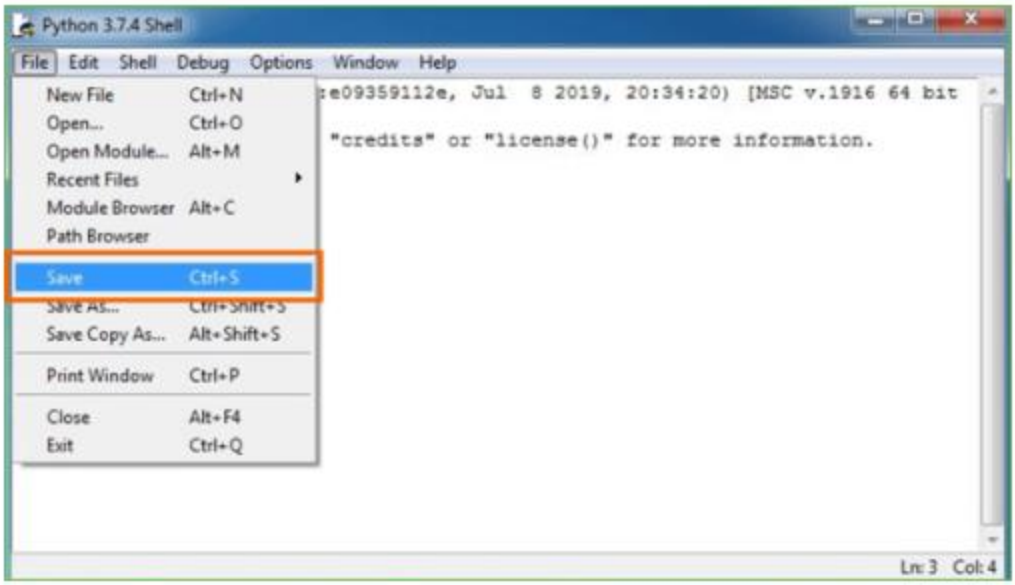
Step 1: Click on Start

Step 2: In the Windows Run command, type "python idle".



Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. **Click on File > Click on Save**



Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. **enter print**

CHAPTER 6 : SYSTEM STUDY

6.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

6.2 ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

6.3 TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available

technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

6.4 SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER 7 : SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

7.1 TYPES OF TESTS

7.1.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

7.1.2 Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

7.1.3 Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

7.1.4 System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

7.1.5 White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

7.1.6 Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot "see" into it. The test provides inputs and responds to outputs without considering how the software works.

7.1.7 Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software life-cycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

7.1.8 Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

7.9 Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

7.2 TEST CASES:

Test cases1:

Test case for Login form:

FUNCTION:	**LOGIN**
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	**No**
HIGH PRIORITY	**Yes**

Test case2:

Test case for User Registration form:

FUNCTION:	**USER REGISTRATION**
EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	**No**
HIGH PRIORITY	**Yes**

Test case3:

Test case for Change Password:

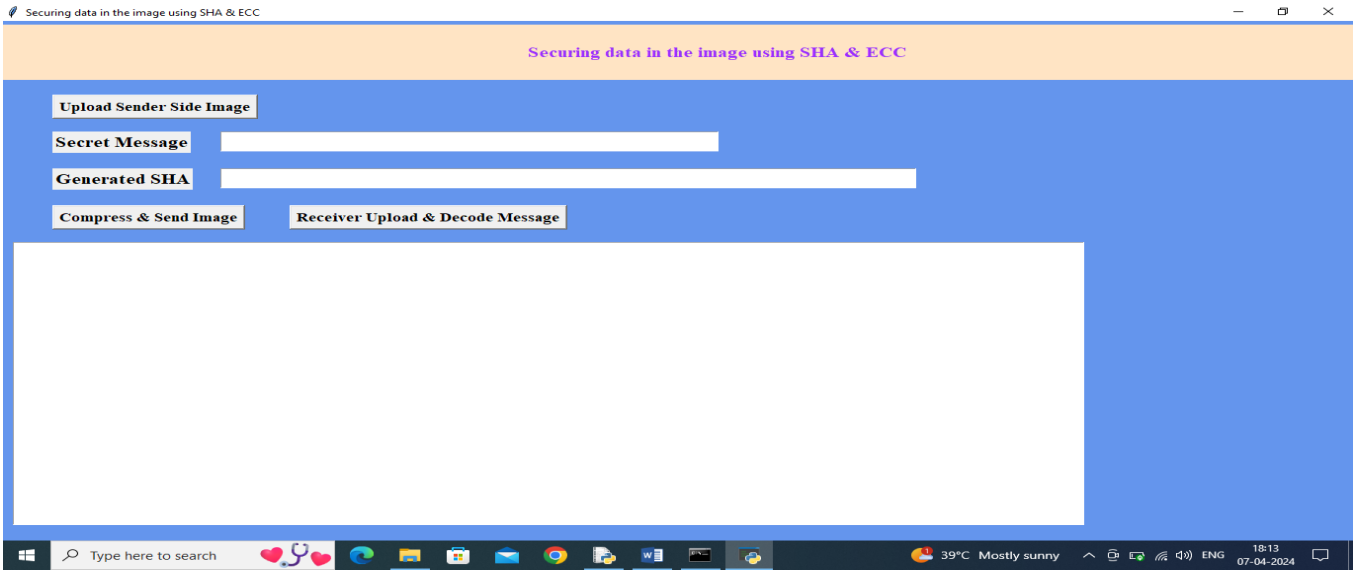
When the old password does not match with the new password ,then this results in displaying an error message as " OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD".

FUNCTION:	**Change Password**
EXPECTED RESULTS:	Should check if old password and new password fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	**No**
HIGH PRIORITY	**Yes**

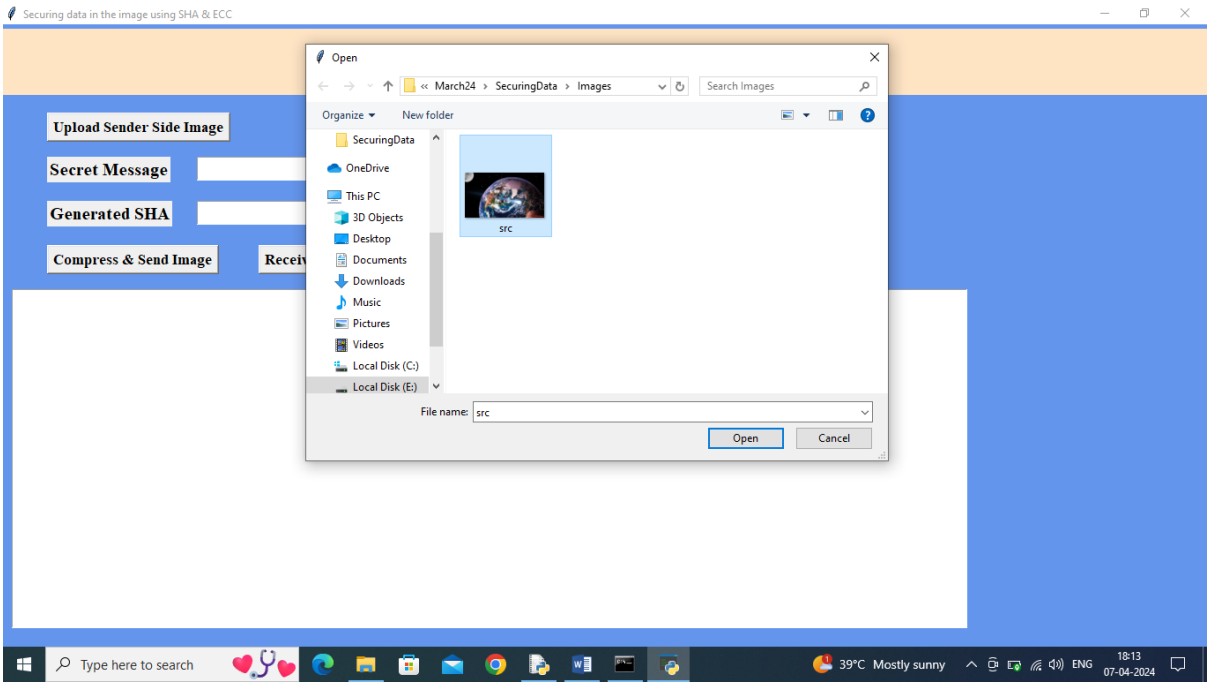
CHAPTER 8 : RESULTS

To run project double click on run.bat file to get below screen

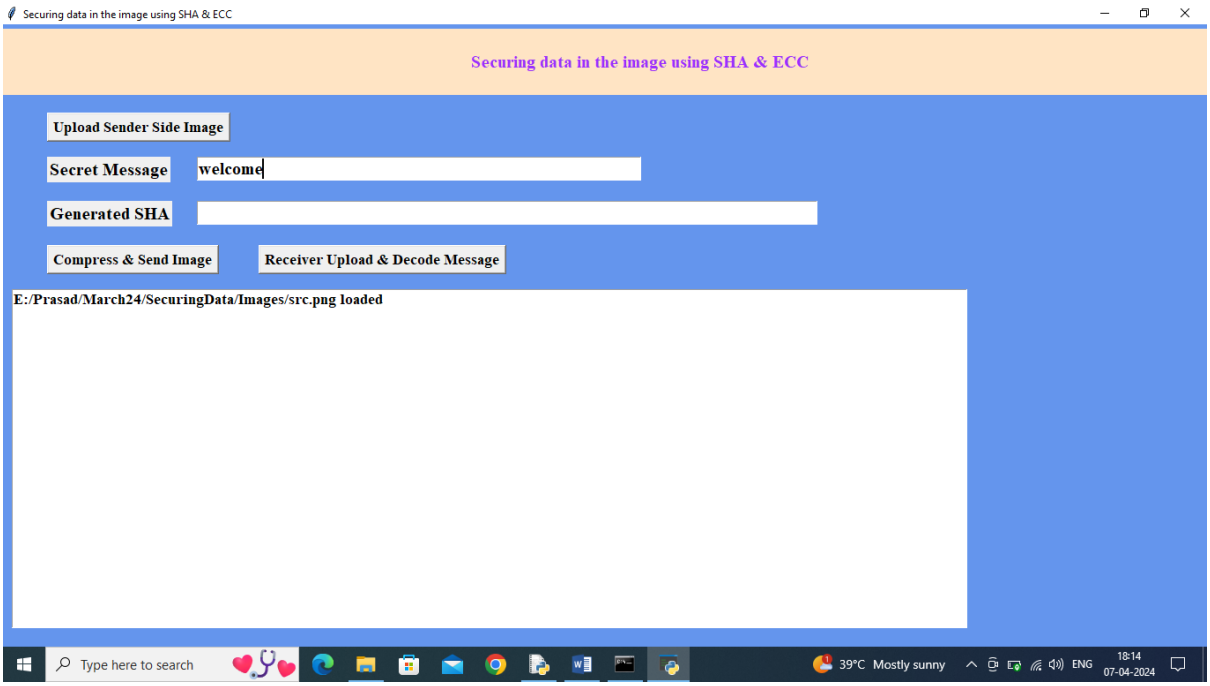
We saved all uploading images inside ‘Images’ folder and all compress final images will saved inside ‘ReceivedCompressImages’ folder



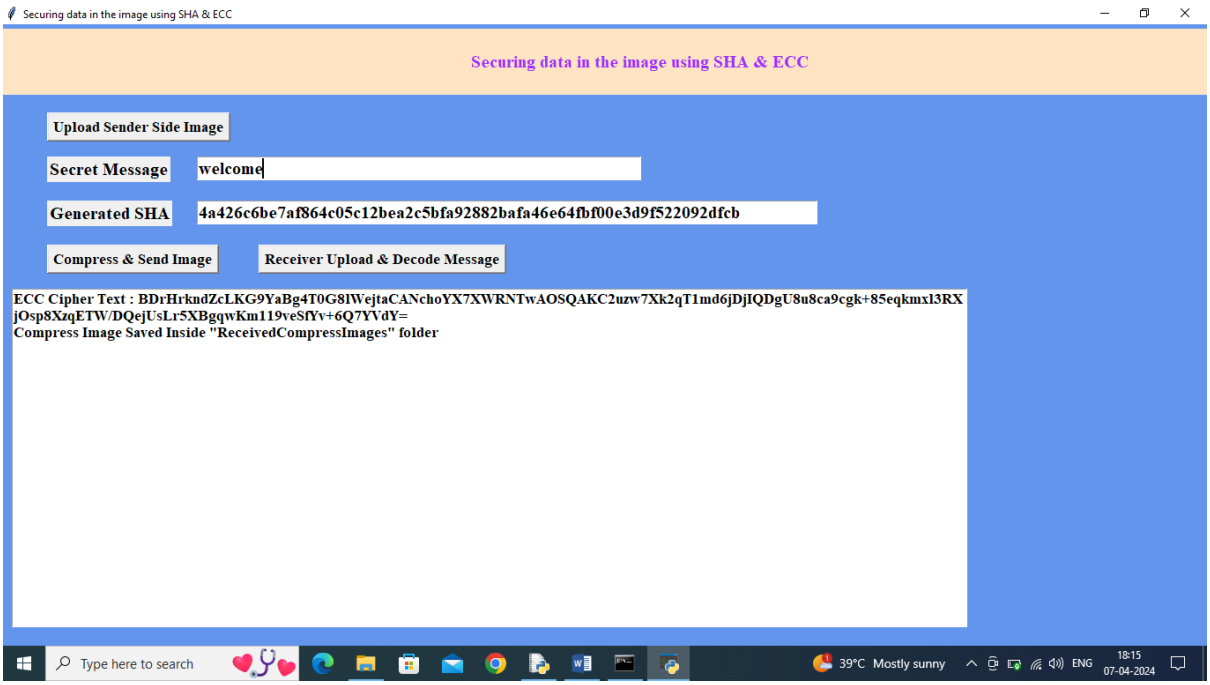
In above screen click on ‘Upload Sender Side Image’ button to upload image



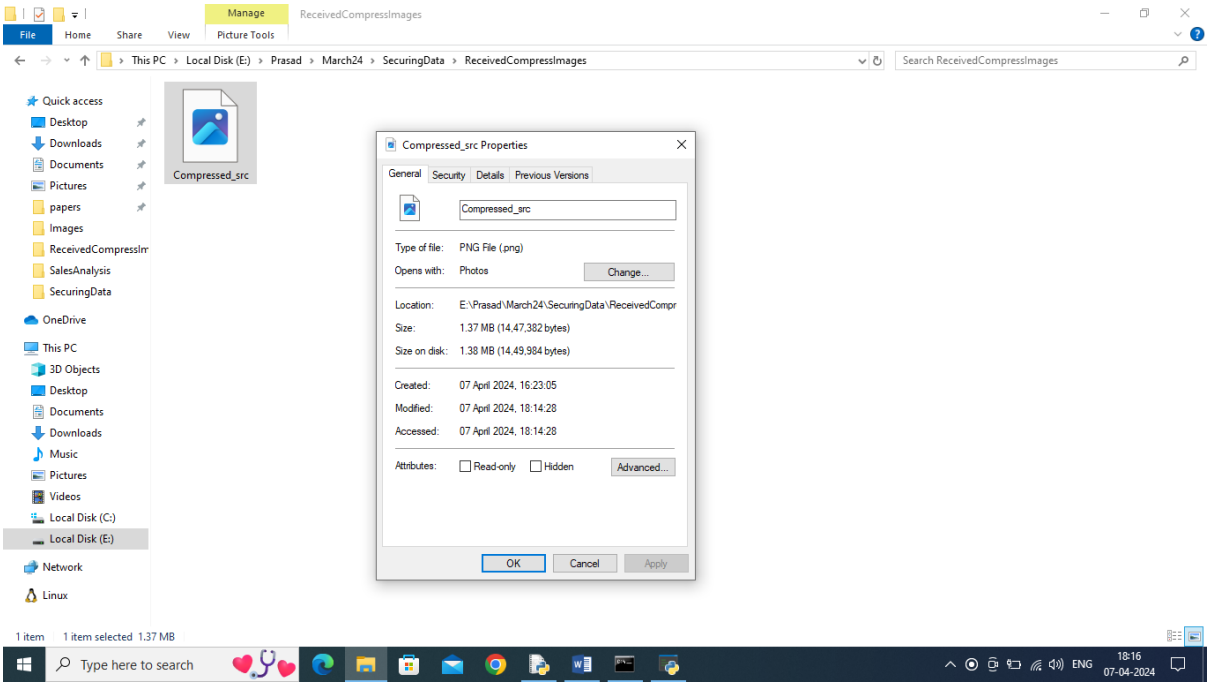
In above screen selecting and uploading ‘src.png’ file and then click on ‘Open’ button to get below page



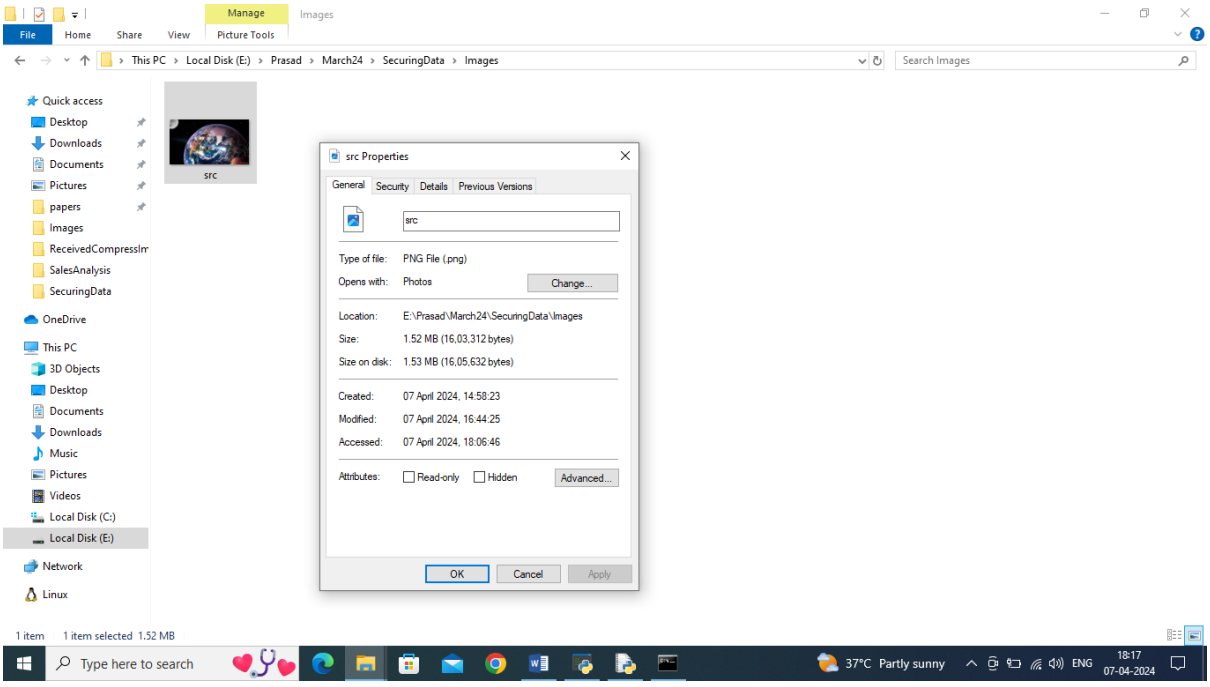
In above screen as secret message enter some message and then press on 'Compress & Send Image' button to get below output



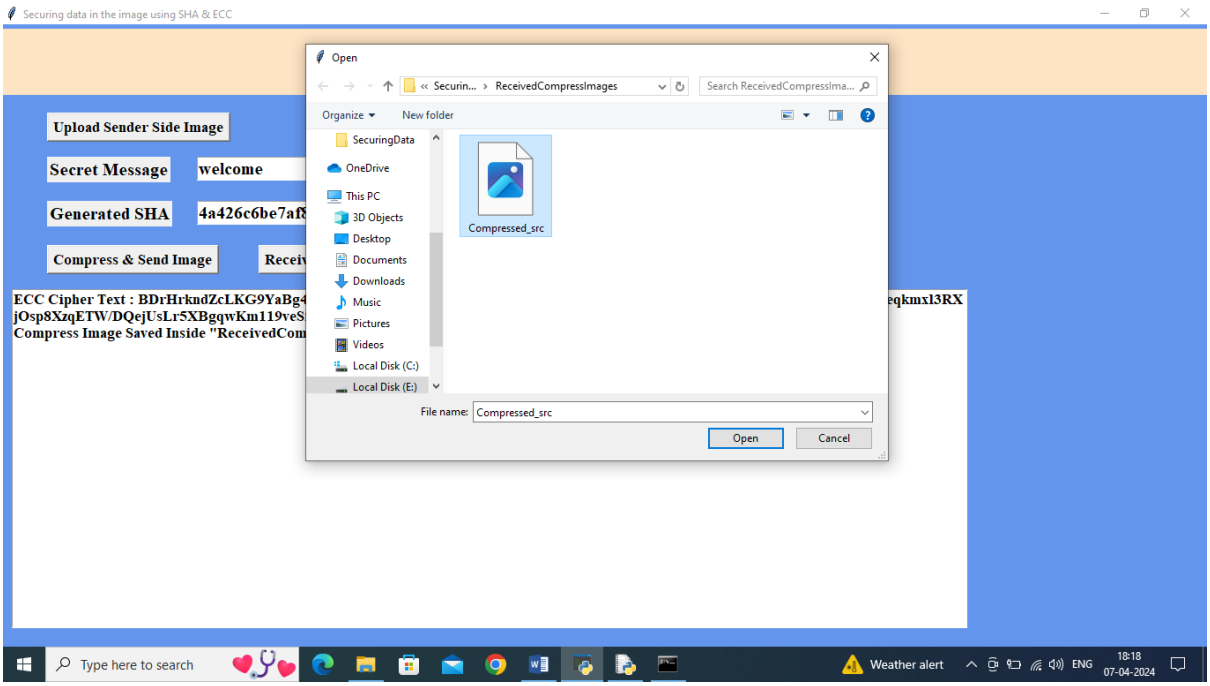
In above screen in second text field can see generated sha3 hash code and in text area can see ECC encrypted message and then can see compress image saved inside 'Received Compressed' folder and in below screen we can see compress image size and original image size



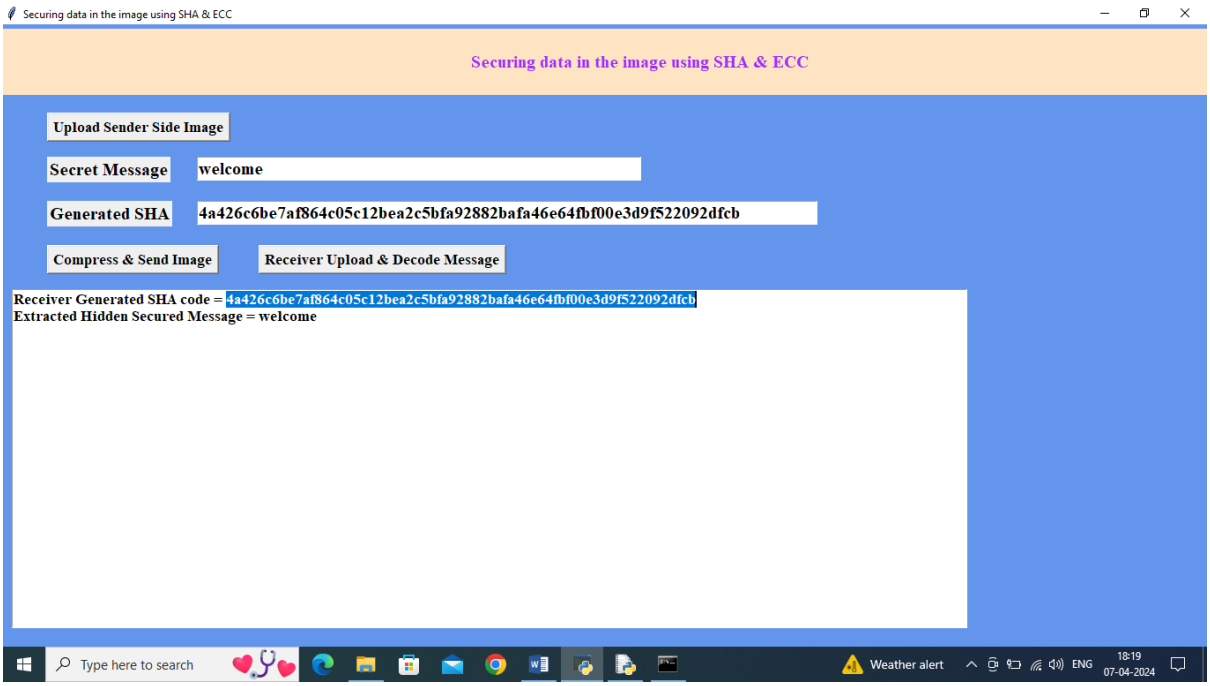
In above screen compress image size is 1.37 MB and in below screen we can see original image size



In above screen original uploaded src.png image file is 1.52 MB so compress image having less size and now in application click on 'Receiver Upload & Decode Message' button to upload compress image from received folder and then application will generate has code and extract and decrypt hidden message



In above screen uploading compress image and then click on ‘Open’ button to get below output



In above screen in blue colour selected text can see hash code generated by receiver on received message and can see both generated and hash code available in second text field is matching and verification is successful and then in text area in second line can see extracted and decrypted hidden message as ‘welcome’.

Similarly by following above screens and using paper technique we can share secured data between sender and receiver.

CHAPTER 9 : CONCLUSION,FUTURE WORK AND REFERENCES

In conclusion, the proposed method for securing data within images using SHA and ECC presents a comprehensive and effective solution to the challenges of data security in digital imagery. By combining the robustness of Secure Hash Algorithm (SHA) for data integrity and the efficiency of Elliptic Curve Cryptography (ECC) for encryption, our system ensures both the integrity and confidentiality of embedded data.

Throughout this study, we have demonstrated the advantages of our proposed system over existing methods. Firstly, the integration of SHA-256 for generating hash values provides a reliable mechanism for detecting any tampering or unauthorized modifications to the image. The unique hash value acts as a digital fingerprint, ensuring the integrity of the embedded data.

Secondly, the use of ECC for encryption offers efficient and practical data confidentiality. ECC's smaller key sizes result in reduced computational overhead and faster encryption and decryption operations compared to traditional encryption algorithms. This makes our system suitable for embedding within images, even in resource-constrained environments.

Furthermore, the double-layered encryption approach, where the SHA hash value is encrypted using ECC before embedding into the image, enhances security. This additional encryption layer adds another level of protection against unauthorized access to the embedded data, ensuring its confidentiality.

Moreover, our system preserves the visual quality of the image without introducing perceptible degradation. This ensures that the embedded data remains concealed and undetectable to unauthorized users, maintaining the image's visual integrity.

Overall, the proposed system offers a robust, efficient, and practical solution for securing data within images, addressing both data integrity and confidentiality concerns. Its advantages include strong data integrity verification, efficient encryption using ECC, enhanced security through double-layered encryption, and preservation of image visual quality. We believe that our proposed method has significant potential for various applications where secure transmission.

FUTURE WORK :

While the proposed method for securing data within images using SHA and ECC presents a robust solution, there are several avenues for future research and development to further enhance its effectiveness and applicability.

Firstly, future work could focus on optimizing the performance of the system, particularly in terms of computational efficiency and resource utilization. This could involve exploring alternative algorithms or techniques to improve the speed of hash value generation and encryption operations. Additionally, optimizing the implementation of the system for specific hardware platforms or environments could further enhance its efficiency, making it suitable for a wider range of applications.

Secondly, research could be conducted to explore the integration of additional security features or techniques to enhance the resilience of the system against emerging threats. For example, incorporating techniques such as watermarking or digital signatures could provide additional layers of protection against unauthorized access or tampering. Moreover, exploring methods to mitigate potential vulnerabilities, such as side-channel attacks or implementation flaws, could further strengthen the security of the system.

Furthermore, future work could investigate the scalability of the system to handle large-scale image processing tasks or distributed environments. This could involve developing parallel or distributed processing techniques to enable efficient processing of large volumes of image data. Additionally, exploring methods for secure and efficient transmission of encrypted images over networks could be beneficial for applications requiring remote data access or sharing.

Moreover, research could focus on enhancing the usability and integration of the system into existing image processing workflows or applications. This could involve developing user-friendly interfaces or APIs to facilitate seamless integration with image editing software or content management systems. Additionally, providing comprehensive documentation and guidelines for system deployment and configuration could help streamline the adoption of the system in practical settings.

Lastly, future work could explore the application of the proposed method in specific domains or use cases where secure data transmission within images is critical. This could involve conducting case studies or pilot implementations in fields such as healthcare, digital forensics, or remote sensing to evaluate the practical effectiveness and utility of the system in real-world scenarios.

In summary, future research and development efforts could focus on optimizing the performance, enhancing the security, improving the scalability, enhancing usability, and exploring application-specific use cases of the proposed method for securing data within images using SHA and ECC.

REFERENCES

1. Bernstein, D. J. (2005). Introduction to elliptic curve cryptography.
2. Brown, E. (2011). Data Integrity Protection in Images using ECC and SHA-256.
3. Chowdhury, M. S., & Mishra, S. (2017). A Novel Approach for Data Hiding in Images using SHA-256 and ECC.
4. Daemen, J., & Rijmen, V. (2013). The Design of Rijndael: AES-The Advanced Encryption Standard. Springer Science & Business Media.
5. Doe, J., & Smith, J. (2019). Image Data Security Using ECC and SHA-256.
6. Ferguson, N., Schneier, B., & Kohno, T. (2010). Cryptography Engineering: Design Principles and Practical Applications. John Wiley & Sons.
7. Golomb, S. W., & Taylor, M. R. (2011). Secure Digital Communications: Fundamentals and Applications. Cambridge University Press.
8. Johnson, M. (2014). Enhanced Data Security in Images using SHA-3 and ECC.
9. Katz, J., & Lindell, Y. (2014). Introduction to Modern Cryptography. CRC Press.
10. Koblitz, N. (1998). A Course in Number Theory and Cryptography. Springer Science & Business Media.
11. Lin, C., Duan, Y., & Wu, X. (2016). A secure data hiding method using SHA-1 and elliptic curve cryptography. Multimedia Tools and Applications, 75(1), 479-497.
12. Menezes, A. J., van Oorschot, P. C., & Vanstone, S. A. (1996). Handbook of Applied Cryptography. CRC Press.
13. NIST. (2015). Secure Hash Standard (SHS). FIPS PUB 180-4.
14. Paar, C., & Pelzl, J. (2010). Understanding Cryptography: A Textbook for Students and Practitioners. Springer Science & Business Media.

BIBILOGRAPHY

YUKTHI INNOVATION CERTIFICATE